



Harbinger: Time Series Transformations



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Roadmap

Let's explore practical examples of preprocessing with the **Harbinger** package in R — from smoothing with moving averages to symbolic representations (SAX and XSAX).

01

Smoothing

Moving average smoothing with `mas()`

02

SAX

Symbolic representation with `trans_sax()`

03

XSAX

Extended alphabet representation with `trans_xsax()`

📄 Run the complete examples and explore the source code at: <https://github.com/cefet-rj-dal/harbinger/tree/master/examples>

Moving Average Smoothing

What is it? A smoothing technique that replaces each value in a time series with the average of its neighboring values within a window of size k (the **order** parameter). This reduces local noise while preserving the overall trend.

How it works: For each point t , the smoothed value is the mean of the k surrounding observations. Larger windows produce smoother signals but may blur sharp transitions.

When to use it: Before applying event detectors, to reduce high-frequency noise that could trigger false positives.

Trade-off: Small **order** → more responsive, more noise. Large **order** → smoother, but may mask real change-points.

- ❏ "Hyndman, R. J., Athanasopoulos, G. (2021). Forecasting: Principles and Practice (3rd ed.). OTexts. otexts.com/fpp3 — Chapter 3: Time Series Decomposition."

01

Raw Signal

Noisy time series input

02

Sliding Window

Average computed over k neighbors

03

Smoothed Output

Cleaner signal ready for detection

Set Up the Example: mas()

```
# Install Harbinger (if needed)
# install.packages("harbinger")

# Load packages
library(harbinger)

# Load change-point datasets
data(examples_changepoints)

# Select simple dataset
dataset <- examples_changepoints$simple
head(dataset)

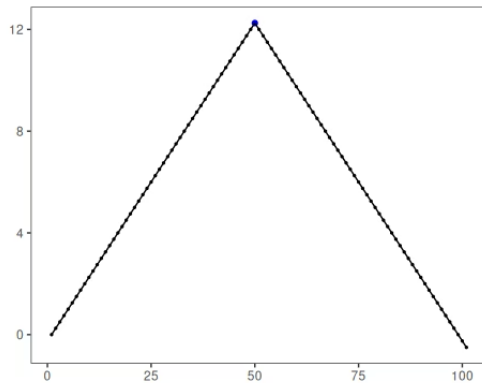
## serie event
## 1 0.00 FALSE
## 2 0.25 FALSE
## 3 0.50 FALSE
## 4 0.75 FALSE
## 5 1.00 FALSE
## 6 1.25 FALSE
```

Step by Step

- Install the `harbinger` package if it is not yet available
- Load the package with `library(harbinger)`
- Use `data(examples_changepoints)` to access the built-in example datasets
- Select the `simple` dataset — it features a clear, well-defined level shift
- `head()` previews the first rows: the `serie` column holds the signal values and `event` holds the change-point markers

Visualize the Original Series

```
# Plot the original time series
har_plot(
  harbinger(),
  dataset$serie,
  event = dataset$event
)
```



Step by step

- `har_plot()` is Harbinger's standard visualization function
- The first argument is a base `harbinger()` object
- `dataset$serie` provides the numeric values of the series
- `event = dataset$event` overlays the event markers on the plot
- Notice where the level shift occurs — this is the pattern the smoother must preserve

❏ **Key concept:** Visually inspecting the raw signal reveals whether the pattern is clean, weak, local, or repeated — before any modeling takes place.

Configure and Apply `mas()`

```
# Calculate smoothers with two window sizes
ma_5 <- mas(dataset$serie, order = 5)
ma_15 <- mas(dataset$serie, order = 15)
```

```
# Inspect the first smoothed values
head(ma_5)
## Time Series: Start=1, End=6, Freq=1
## [1] 0.50 0.75 1.00 1.25 1.50 1.75

head(ma_15)
## Time Series: Start=1, End=6, Freq=1
## [1] 1.75 2.00 2.25 2.50 2.75 3.00
```

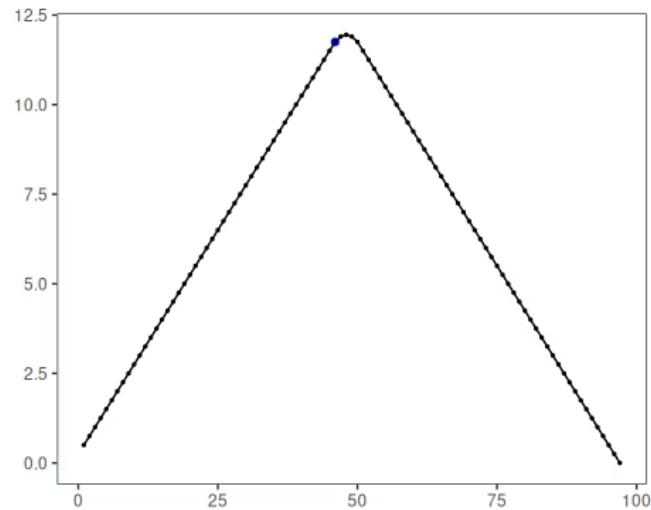
Step by step

- `mas(serie, order = k)` computes the simple moving average with a window of size `k`
- `order = 5`: small window — preserves more local detail
- `order = 15`: large window — more aggressive smoothing, higher values
- Notice that the first values differ: larger windows shift the series further in time
- The return value is an R `ts` (time series) object

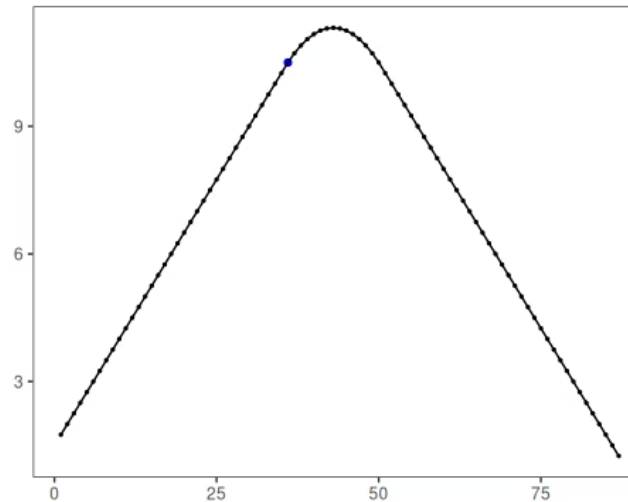
📌 **Key concept:** The `order` parameter controls the trade-off between smoothness and temporal resolution — larger windows eliminate more noise but lose fine-grained detail.

Visualizing the Smoothed Series

```
# Plot 5-point moving average
har_plot(
  harbinger(),
  as.numeric(ma_5),
  event = dataset$event[5:length(dataset$event)]
)
```



```
# Plot 15-point moving average
har_plot(
  harbinger(),
  as.numeric(ma_15),
  event =
    dataset$event[15:length(dataset$event)]
)
```



Step by step

- `as.numeric()` converts the `ts` object into a plain numeric vector
- The event vector is trimmed with `[k:length(...)]` to align with the smoothed series, which has `k-1` fewer points
- Compare the two plots: `order=5` keeps the transition clearly visible; `order=15` dilutes it further
- Smoothing is often used as a preprocessing step before applying change-point detectors

📌 **Key concept:** Smoothing before detection reduces false positives caused by local noise — but windows that are too large can mask genuine change points.

SAX – Symbolic Aggregate approXimation

What is it? SAX is a symbolic representation of time series that converts a continuous numeric signal into a sequence of discrete symbols (letters), enabling pattern matching, indexing, and dimensionality reduction.

How it works: The series is first z-normalized (zero mean, unit variance). It is then divided into equal-length segments, each replaced by its mean (PAA — Piecewise Aggregate Approximation). Finally, each PAA value is mapped to a symbol based on breakpoints derived from a Gaussian distribution, producing a string of **alpha** symbols.

Key parameter: **alpha** — the alphabet size. Larger alphabets capture more detail; smaller alphabets yield more compact representations.

When to use it: When you need to compare, cluster, or index time series by shape rather than exact values.

- ❏ "Lin, J., Keogh, E., Wei, L., Lonardi, S. (2007). Experiencing SAX: a novel symbolic representation of time series. Data Mining and Knowledge Discovery, 15(2), 107–144. doi:10.1007/s10618-007-0064-z"

01

Z-Normalization

zero mean, unit variance

02

PAA Segmentation

divide into equal windows, compute means

03

Symbol Mapping

assign letters via Gaussian breakpoints → string output

Setting Up the Example: `trans_sax()`

```
# Load required packages
library(daltoolbox)
library(harbinger)

# Load motif datasets
data(examples_motifs)

# Select the simple dataset
dataset <- examples_motifs$simple
head(dataset)

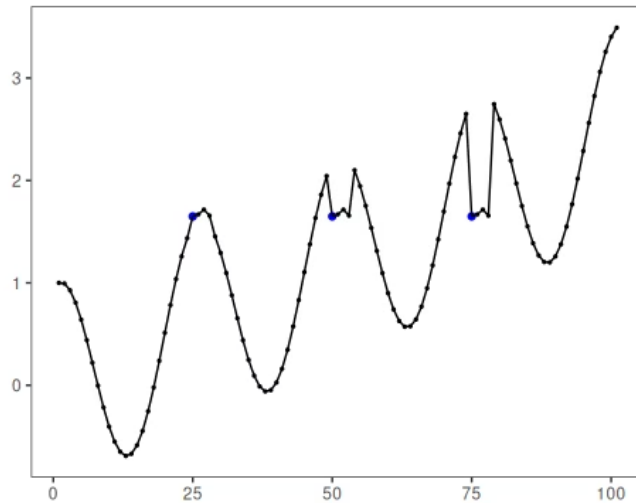
## serie  event
## 1  1.0000000 FALSE
## 2  0.9939124 FALSE
## 3  0.9275826 FALSE
## 4  0.8066889 FALSE
## 5  0.6403023 FALSE
```

Step-by-step

- SAX also requires the `daltoolbox` package — load it before `harbinger`
- Use `data(examples_motifs)` to load time series with repeated patterns (motifs)
- The `simple` dataset contains an oscillatory series with values between -1 and 1
- The `event` column marks positions of interest for later validation

Visualize and Configure `trans_sax()`

```
# Plot the original series
har_plot(
  harbinger(),
  dataset$serie,
  event = dataset$event
)
```



```
# Configure the SAX transformer
model <- trans_sax(alpha = 8)

# Fit and transform the series
model <- fit(model, dataset$serie)
sax_series <- transform(model, dataset$serie)

# Inspect the first symbols
head(sax_series, 20)
## [1] "D" "D" "D" "C" "C" "B" "B" "B"
##  "A" "A" "A" "A" "A" "A" "A" "A"
##  "A" "A" "B" "B"
```

Step-by-step

- `trans_sax(alpha = 8)` creates a transformer with an 8-letter alphabet (A–H)
- `fit(model, serie)` estimates the discretization breakpoints via z-normalization
- `transform(model, serie)` applies the transformation and returns symbols
- High values (~1.0) map to "D"; low values (~-0.7) map to "A"
- The result is a character sequence — ideal for motif and discord analysis

📌 **Key concept:** The `alpha` parameter defines the alphabet size: more letters means a finer partition of the numeric values.

Mapping Numeric Values to SAX Symbols

```
# Create a comparison table
comparison <- data.frame(
  idx = seq_along(dataset$serie),
  value = dataset$serie,
  sax = sax_series
)
head(comparison, 20)
```

```
## idx    value sax
## 1  1  1.00000000 D
## 2  2  0.99391242 D
## 3  3  0.92758256 D
## 4  4  0.80668887 C
## 5  5  0.64030231 C
## 6  6  0.44032236 B
## 7  7  0.22073720 B
## 8  8 -0.00324606 B
## 9  9 -0.21614684 A
## 10 10 -0.40317362 A
```

```
## 11 11 -0.56320704 A
## 12 12 -0.68454710 A
## 13 13 -0.75440651 A
## 14 14 -0.76940314 A
## 15 15 -0.72896904 A
## 16 16 -0.63560460 A
## 17 17 -0.49399165 A
## 18 18 -0.31993155 A
## 19 19  0.23920420 B
## 20 20  0.51260215 B
```

Step by Step

- Build a `data.frame` with the index, original value, and SAX symbol side by side
- Notice the transition: decreasing values from ~1.0 to -0.7 map as `D → C → B → A`
- Starting at index 19, the series rises again: `A → B`
- This correspondence confirms that SAX preserves the coarse shape of the signal

📖 **Reference:** Lin et al. (2007). A symbolic representation of time series. *Data Mining and Knowledge Discovery*, 15, 107–144.

XSAX – Extended Symbolic Representation

What is it? XSAX is an extension of SAX that uses a larger alphanumeric alphabet (digits 0–9 and letters A–F, up to 16 symbols), enabling finer-grained symbolic partitioning of the signal.

How it works: Like SAX, XSAX z-normalizes the series and applies PAA segmentation. The key difference is the alphabet: instead of a small set of letters (e.g., a–d), XSAX maps values to a 16-symbol alphanumeric set, capturing more intermediate levels of the signal.

Key parameter: `alpha = 16` — uses the full extended alphabet. Can be reduced for coarser representations.

SAX vs. XSAX: SAX is ideal for coarse shape matching; XSAX is better when finer amplitude distinctions matter (e.g., detecting subtle motifs or gradual drifts).

When to use it: When SAX's alphabet is too coarse and you need higher symbolic resolution without returning to raw numeric values.

SAX

small alphabet (e.g., a–d)

coarse partitioning

compact representation

XSAX

large alphabet (0–9, A–F)

fine partitioning

higher symbolic resolution

❏ "Ogasawara, E., Salles, R., Porto, F., Pacitti, E. (2025). Event Detection in Time Series. Springer. doi:10.1007/978-3-031-75941-3"

Configure and Apply `trans_xsax()`

```
# Reuse the same motifs dataset
# dataset <- examples_motifs$simple

# Configure the XSAX transformer
model <- trans_xsax(alpha = 16)

# Fit and transform
model <- fit(model, dataset$serie)
xsax_series <- transform(model,
dataset$serie)
```

```
# Inspect the first symbols
head(xsax_series, 20)
## [1] "6" "6" "6" "5" "4" "3" "2" "2"
##  "1" "0" "0" "0" "0" "0" "0" "0"
##  "1" "1" "2" "3"
```

Step by Step

- XSAX follows the same logic as SAX: it normalizes and discretizes the series
- `trans_xsax(alpha = 16)` uses a larger alphanumeric alphabet (0–9, A–F)
- The workflow is identical: `fit()` → `transform()`
- High values (~1.0) map to "6"; low values (~-0.7) map to "0"
- Compared to SAX, XSAX distinguishes more intermediate levels (e.g., 3, 4, 5 vs. just B, C)

Mapping Numeric Values to XSAX Symbols

```
# Build a comparison table: numeric values vs.  
XSAX symbols
```

```
comparison <- data.frame(  
  idx = seq_along(dataset$serie),  
  value = dataset$serie,  
  xsax = xsax_series  
)  
head(comparison, 20)
```

```
## idx    value xsax  
## 1  1  1.00000000  6  
## 2  2  0.99391242  6  
## 3  3  0.92758256  6  
## 4  4  0.80668887  5  
## 5  5  0.64030231  4  
## 6  6  0.44032236  3  
## 7  7  0.22073720  2  
## 8  8 -0.00324606  2  
## 9  9 -0.21614684  1  
## 10 10 -0.40317362  0
```

```
## 11 11 -0.56320704  0  
## 12 12 -0.68454710  0  
## 13 13 -0.75440651  0  
## 14 14 -0.76940314  0  
## 15 15 -0.72896904  0  
## 16 16 -0.63560460  0  
## 17 17 -0.49399165  0  
## 18 18 -0.31993155  0  
## 19 19  0.23920420  2  
## 20 20  0.51260215  3
```

Step-by-step

- The descent from ~ 1.0 to ~ -0.7 produces the sequence $6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$
- With SAX ($\alpha=8$), the same descent yielded only $D \rightarrow C \rightarrow B \rightarrow A$ — just 4 levels
- XSAX captures 7 distinct levels in the same range, delivering much finer symbolic resolution
- Especially valuable when motifs or discords differ in amplitude in subtle ways

📖 **Reference:** Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025. doi:10.1007/978-3-031-75941-3

Summary: Choosing the Right Transformation

`mas()` – Smoothing

- Reduces local noise before detectors
- Parameter: `order` (window size)
- Output: smoothed numeric series (`ts`)
- Use when the signal has high local variance

`trans_sax()` – SAX

- Discretizes into a compact alphabet (letters)
- Parameter: `alpha` (alphabet size)
- Output: character vector (A–H)
- Use for motif and discord analysis

`trans_xsax()` – XSAX

- Larger alphanumeric alphabet than SAX
- Parameter: `alpha` (up to 36 symbols)
- Output: character vector (0–9, A–Z)
- Use when SAX loses important resolution

📌 **Recommended learning order:** Start with `mas()` (intuitive to visualize), then `trans_sax()`, and finally `trans_xsax()` for higher symbolic resolution.

Additional Sources

Core Textbook

Ogasawara, E., Salles, R., Porto, F., Pacitti, E. *Event Detection in Time Series*. Springer, 2025.
doi:10.1007/978-3-031-75941-3

SAX – Original Paper

Lin, J., Keogh, E., Wei, L., Lonardi, S. (2007). Experiencing SAX: a novel symbolic representation of time series. *Data Mining and Knowledge Discovery*, 15(2), 107–144.
doi:10.1007/s10618-007-0064-z

Moving Average Smoothing

Hyndman, R. J., Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Available at: otexts.com/fpp3 (Chapter 3: Time series decomposition)

tspredict on CRAN

A framework for time series prediction with integrated tuning. Supports filtering, decomposition, outlier detection, and data augmentation — useful for exploring additional preprocessing beyond `mas()`. Available at: cran.r-project.org/package=tspredict

Harbinger on GitHub

Explore examples, open issues, contribute, and follow development on the official GitHub repository: github.com/cefet-rj-dal/harbinger